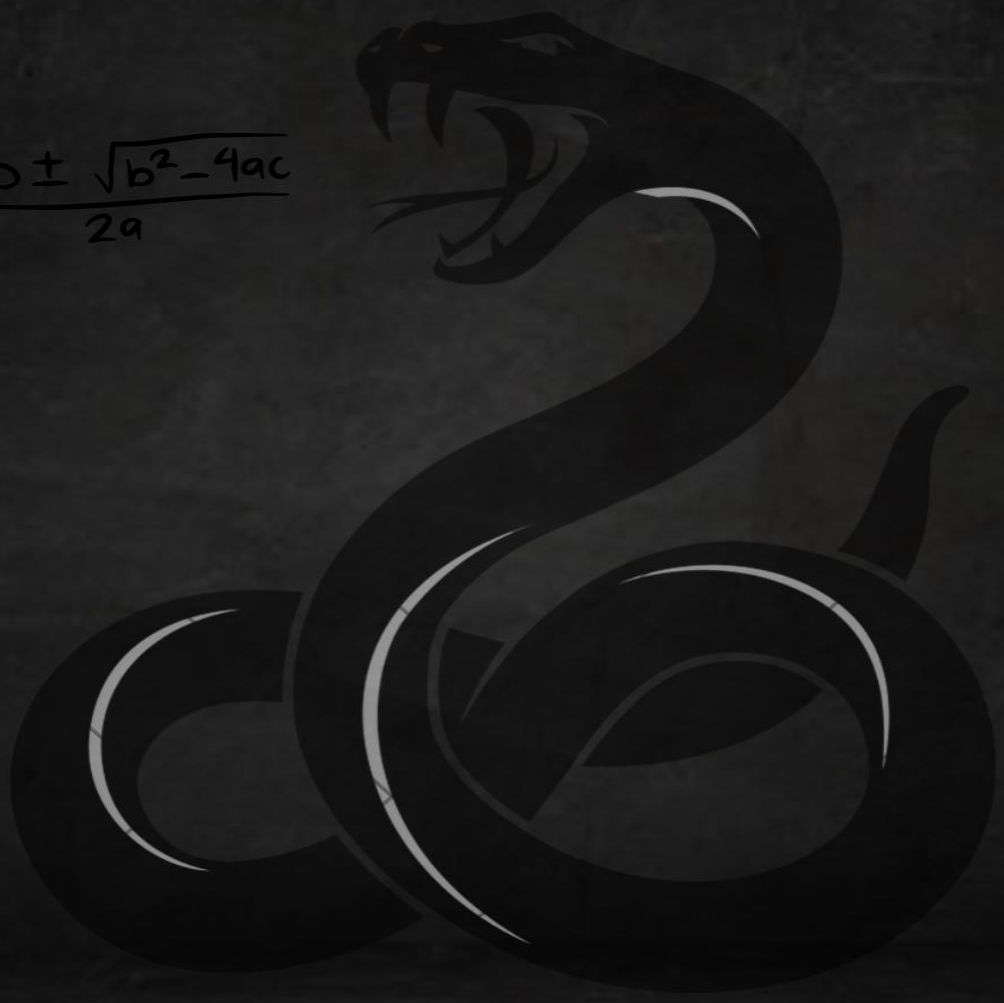
AL_SHMAL
UNIVERSITY

Numerical Analysis
Python (sympy)

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$



Abdulmoin Hablas
Dr.Ahmed steif

Numerical Analysis – Python (Sympy)

1 مقدمة

التحليل العددي هو فرع مهم من الرياضيات يعنى بإيجاد حلول تقريبية للمشكلات الرياضية باستخدام خوارزميات عددية. يهدف هذا المشروع إلى استخدام مكتبة SymPy في بايثون لتنفيذ مجموعة من العمليات الأساسية في التحليل العددي مثل حل المعادلات، الاشتقاق، التكامل، والاستيفاء. سيشمل الشرح في هذا الملف كيفية تطبيق هذه الأساليب باستخدام SymPy وكيفية الاستفادة منها في البرمجة لحل المسائل الرياضية.

2 نظرة عامة على التحليل العددي:

2.1 تعريف التحليل العددي:

التحليل العددي هو فرع من الرياضيات التطبيقية التي تركز على تطوير وتنفيذ الطرق العددية لإيجاد حلول تقريبية للمشاكل الرياضية المعقدة التي يصعب حلها باستخدام الطرق التحليلية التقليدية. يتضمن هذا المجال استخدام الخوارزميات والأساليب العددية لتقريب حلول المعادلات الرياضية، سواء كانت جبرية أو تفاضلية، بطرق تضمن دقة مقبولة وكفاءة حسابية.

2.2 أهميته وتطبيقاته:

تكمن أهمية التحليل العددي في قدرته على التعامل مع المعادلات التي لا يمكن إيجاد حلولها التحليلية بسهولة، مثل المعادلات التي تتضمن دوال غير خطية أو تفاضلية معقدة. كما يستخدم في مجالات متعددة مثل علوم الحاسوب، الهندسة، الفيزياء، والاقتصاد، حيث يساهم في حل المشكلات التي تتطلب حسابات دقيقة وسريعة.

تتراوح تطبيقات التحليل العددي بين حل المعادلات الجبرية، التقدير العددي للتكاملات، التفاضل العددي، واستيفاء البيانات. كما أن التطورات الحديثة في هذا المجال قد أسهمت في تحسين كفاءة الخوارزميات المستخدمة وحسابات النماذج الرياضية المعقدة، مما يسهم في تحقيق حلول دقيقة وعملية للمشكلات العلمية والهندسية.

3 نظرة عامة على مكتبة SymPy:

تُعد مكتبة SymPy إحدى المكتبات البرمجية مفتوحة المصدر المخصصة للرياضيات الرمزية، وهي مكتوبة بلغة البرمجة Python. تهدف هذه المكتبة إلى توفير أدوات قوية لتحليل المعادلات الرياضية وإجراء العمليات الرمزية على الدوال والمعادلات الرياضية، وذلك بدلاً من الاعتماد على العمليات العددية المباشرة. توفر SymPy إمكانيات واسعة في مجالات مختلفة من الرياضيات مثل الجبر، والتفاضل والتكامل، والعديد من التطبيقات الرياضية الأخرى.

3.1 الخصائص الرئيسية لمكتبة SymPy:

3.1.1 حل المعادلات الجبرية: تمكّن المكتبة المستخدم من حل المعادلات الجبرية، سواء كانت خطية أو غير خطية، بشكل رمزي، مما يسمح بالحصول على حلول دقيقة تُعبر عن المعادلة بدلالة المتغيرات.

3.1.2 التفاضل والتكامل الرمزي: تعدّ SymPy أداة قوية في إجراء العمليات التفاضلية والتكاملية بشكل رمزي، حيث يمكن حساب المشتقات والتكاملات بدقة ودون الحاجة لتحويل المعادلات إلى قيم عددية.

3.1.3 المصفوفات: تدعم المكتبة العمليات الرياضية المعقدة على المصفوفات، بما في ذلك العمليات الجبرية مثل القيم الذاتية للمصفوفات، والمصفوفات المقلوبة، وغيرها.

3.1.4 الرسوم البيانية: على الرغم من أن SymPy تركز بشكل رئيسي على الرياضيات الرمزية، إلا أنها توفر أدوات لتمثيل الدوال والمعادلات بيانيًا، مما يسهل فهم وتحليل سلوك الدوال الرياضية.

3.1.5 دعم تنسيقات متعددة: يمكن تحويل النتائج إلى تنسيقات مختلفة مثل LaTeX و MathML، مما يسهل تضمينها في المستندات العلمية والتقارير.

3.2 التطبيقات الأكاديمية والصناعية:

تُستخدم مكتبة SymPy على نطاق واسع في العديد من التطبيقات الأكاديمية والصناعية، فهي تعد أداة أساسية في مجالات الرياضيات التطبيقية، والهندسة، والفيزياء، والاقتصاد، وعلم البيانات. كما أنها تسهم في تطوير الخوارزميات الرياضية وتحليل النماذج الرياضية المعقدة، مما يجعلها من الأدوات الأساسية في البحث العلمي والتعليم.

تعتبر SymPy منصة مرنة للباحثين والطلاب وأيضًا المهندسين الذين يسعون إلى التعامل مع المعادلات الرياضية بشكل دقيق وفعّال، بما يتيح لهم اكتساب حلول دقيقة وموثوقة لمشاكل رياضية متنوعة.

4 حل المعادلات الرياضية باستخدام SymPy:

يعد حل المعادلات الرياضية جزءًا أساسيًا في العديد من التطبيقات العلمية والهندسية. تقدم مكتبة SymPy أدوات فعّالة لحل مجموعة واسعة من المعادلات، سواء كانت خطية أو غير خطية. في هذا القسم، سنستعرض كيفية استخدام SymPy لحل هذه المعادلات بشكل دقيق وفعّال.

4.1 حل المعادلات الخطية:

المعادلات الخطية هي معادلات من الدرجة الأولى، والتي يمكن تمثيلها بالصورة العامة:

- $ax + b = 0$

حيث a وb هما ثوابت وx هو المتغير المجهول. في SymPy، يمكن حل المعادلات الخطية بسهولة باستخدام دالة solve () التي تقبل المعادلة وتعيد الحلول الممكنة.

- مثال:

```
from sympy import symbols, Eq, solve
# تعريف المتغير
x = symbols('x')
# المعادلة
equation = Eq(2*x + 3, 0)
# حل المعادلة
solution = solve(equation, x)
print(solution)
```

النتيجة: $x = \frac{-2}{3}$

يمكن لـ SymPy أيضًا التعامل مع مجموعة من المعادلات الخطية، سواء كانت تحتوي على أكثر من معادلة أو متغير.

```
from sympy import symbols, Eq, solve
# تعريف المتغيرات
x, y = symbols('x y')
# تعريف المعادلات
equation1 = Eq(2*x + 3*y, 5)
equation2 = Eq(4*x - y, 3)
# حل المعادلات
solutions = solve((equation1, equation2), (x, y))
Print(f"حل المعادلات هو: {solutions}")
```

والنتيجة:

$x = 1 , y = 1$

4.2 حل المعادلات غير الخطية:

المعادلات غير الخطية هي معادلات تحتوي على درجات أعلى من المتغير أو دوال غير خطية مثل الجذور التربيعية أو الأسس. يمكن حل المعادلات غير الخطية باستخدام SymPy بنفس الطريقة المستخدمة في المعادلات الخطية.

- مثال حل معادلة غير خطية من الدرجة الثانية:

$$x^2 - 4 = 0$$

```
# المعادلة غير الخطية
equation = Eq(x**2 - 4, 0)
# حل المعادلة
solutions = solve(equation, x)
print(solutions)
```

النتيجة: $x = 2, -2$

يمكن لـ SymPy حل معادلات غير خطية متعددة المتغيرات، بما في ذلك المعادلات التي تحتوي على دوال مثل الجذور، الأسس، والدوال المثلثية.

5 الاشتقاق والتفاضل:

الاشتقاق هو عملية رياضية تهدف إلى دراسة كيفية تغير الدالة عند نقطة معينة. في التحليل العددي، يستخدم الاشتقاق في العديد من التطبيقات مثل دراسة سلوك الدوال، إيجاد النقاط الحرجة، والتقريب باستخدام الأساليب العددية.

تعتبر مكتبة SymPy من الأدوات القوية في Python التي تتيح للمستخدم إجراء العمليات التفاضلية بسهولة وفعالية. توفر هذه المكتبة وظائف لاحتساب المشتقات للدوال الرياضية، سواء كانت دوالًا بسيطة أو دوالًا متعددة المتغيرات.

5.1 اشتقاق دالة بمتغير واحد:

يمكننا حساب المشتقة الأولى لدالة رياضية باستخدام diff () في مكتبة SymPy.

مثال حساب مشتقة الدالة:

$$f(x) = x^2 + 3x + 5$$

```
import sympy as sp
# تعريف المتغير
x = sp.symbols('x')
# تعريف الدالة
f = x**2 + 3*x + 5
# حساب المشتقة
f_prime = sp.diff(f, x)
print(f_prime)
```

- قمنا بتعريف المتغير x باستخدام sp.symbols ().
- ثم عرّفنا الدالة
- استخدمنا sp.diff () لحساب المشتقة الأولى لهذه الدالة.

النتيجة:

$$f'(x) = 2x + 3$$

5.2 اشتقاق دالة بعدة متغيرات:

عند العمل مع دوال متعددة المتغيرات، يمكننا حساب المشتقات الجزئية بالنسبة لكل متغير باستخدام نفس الدالة **diff** (). إذا كانت لدينا دالة ذات متغيرين مثل $f(x, y) = x^2 + y^2 + 2xy$

نحسب المشتقات الجزئية بالنسبة ل x و y:

```
x, y = sp.symbols('x y')
f = x*2 + y*2 + 2*x*y
f_x = sp.diff(f, x)
f_y = sp.diff(f, y)
print(f_x)
print(f_y)
```

- قمنا بتعريف المتغيرات
- عرفنا الدالة
- استخدمنا sp.diff () لحساب المشتقة الجزئية بالنسبة ل x و y.

النتيجة:

$$f\ (x) = 2x + 2y$$

$$f\ (y) = 2y + 2x$$

5.3 المشتقات العليا:

حساب المشتقات العليا لدالة معينة باستخدام diff () مع تحديد عدد مرات الاشتقاق. على سبيل المثال، لحساب المشتقة الثانية للدالة

$$f\ (x) = x^3 - 3x + 1$$

```
# تعريف الدالة
f = x**3 - 3*x + 1
# حساب المشتقة الثانية
f_double_prime = sp.diff(f, x, 2)
print(f_double_prime)
```

النتيجة:

$$f''(x) = 6x$$

6 التكامل:

التكامل هو عملية رياضية تهدف إلى حساب المساحة تحت منحنى دالة معينة. يُعتبر التكامل عكس عملية الاشتقاق، وهو جزء أساسي من التحليل العددي نظرًا لتطبيقاته الواسعة في الفيزياء، الهندسة، والعديد من المجالات الأخرى.

تُتيح مكتبة SymPy أدوات قوية لحساب التكاملات المحدودة وغير المحدودة بسهولة ودقة عالية.

6.1 التكامل غير المحدد:

التكامل غير المحدد هو إيجاد الدالة الأصلية (antiderivative) لدالة معينة. على سبيل المثال، إذا أردنا حساب التكامل غير المحدد للدالة $f(x) = x^2 + 3x + 5$:

```
import sympy as sp
# تعريف المتغير
x = sp.symbols('x')
# تعريف الدالة
f = x**2 + 3*x + 5
# حساب التكامل غير المحدود
F = sp.integrate(f, x)
print(F)
```

- عرفنا المتغير x
- عرفنا الدالة
- استخدمنا sp.integrate () لحساب التكامل غير المحدود.

النتيجة: $F(x) = \frac{x^3}{3} + \frac{3x^3}{2} + 5x + 5 + c$

6.2 التكامل المحدد:

لتكامل المحدد هو إيجاد القيمة العددية للمساحة تحت منحنى دالة بين نقطتين محددتين، ولحساب التكامل المحدد للدالة $F(x) = x^2 + 3x + 5$ بين $[1,3]$، نستخدم الكود التالي:

```
import sympy as sp
# تعريف المتغير
x = sp.symbols('x')
# تعريف الدالة
f = x**2 + 3*x + 5
# حساب التكامل غير المحدود
F_limited = sp.integrate(f, (x, 1, 3))
print(F)
```

- حددنا حدود التكامل $[1,3]$
- استخدمنا الشكل الثاني لدالة sp.integrate () الذي يقبل حدود التكامل.

النتيجة: $\int_1^3 x^2 + 3x + 5 \ dx$

7 الاستيفاء Interpolation

الاستيفاء هو إحدى طرق التحليل العددي التي تهدف إلى إيجاد دالة تقريبية تمر بمجموعة من النقاط المعروفة. يُستخدم الاستيفاء في التنبؤ بالقيم غير المعروفة بناءً على البيانات الموجودة. مكتبة SymPy توفر أدوات لحساب الاستيفاء باستخدام طرق مختلفة مثل طريقة لاغرانج وطريقة نيوتن.

7.1 استيفاء لاغرانج Lagrange Interpolation

طريقة لاغرانج تُستخدم لإيجاد دالة تقريبية تمر بمجموعة من النقاط المعطاة، الشكل العام لاستيفاء لاغرانج هو: $p(x) = \sum_{i=0}^{n} L_i(x).y_i$

حيث: $L_i(x) = \prod_{\substack{i=0 \\ j\neq i}}^{n} \frac{x-x_i}{x_i-x_j}$

طريقة الحل:

- نحسب $L_i(x)$ لكل نقطة i
- نضرب $L_i(x)$ بالقيمة المقابلة y_i
- نجمع الحدود الناتجة لإيجاد $p(x)$

```
import sympy as sp
# تعريف النقاط
x_points = [0, 1, 2]
y_points = [1, 3, 7]
# تعريف المتغير
x = sp.symbols('x')
# حساب دالة لاغرانج
def lagrange_interpolation(x_points, y_points):
    n = len(x_points)
    P = 0
    for i in range(n):
        L = 1
        for j in range(n):
            if i != j:
                L *= (x - x_points[j]) / (x_points[i] - x_points[j])
        P += L * y_points[i]
    return sp.simplify(P)
# استخراج النتيجة
P = lagrange_interpolation(x_points, y_points)
print("دالة الاستيفاء باستخدام لاغرانج:")
```

7.2 استيفاء نيوتن Newton Interpolation

طريقة نيوتن تعتمد على الفروق وهي مفضلة بسبب قدرتها على إضافة النقاط بسهولة دون إعادة الحساب.

7.2.1 الفروق الأمامية (Forward Differences):

تُستخدم عندما تكون النقاط متساوية التباعد، وشكل الدالة هو:

$$p(x) = y_0 + \ r\Delta_{y_0} + \frac{r(r-1)}{2!}\Delta^{\,2}y_0 + \cdots$$

حيث:

- الفرق الأمامي الأول: $\Delta_{y_0} = y_1 - y_0$
- الفرق الأمامي الثاني: $\Delta^2{}_{y_0} = \Delta y_1 - \Delta y_0$
- $r = \frac{x-x_0}{h}$ ، h: الفرق الثابت بين قيم x

طريقة الحل:

1. بناء جدول الفروق الأمامية
2. حساب الفرق الأمامي الأول، ثم الثاني، وهكذا
3. استخدام القيم المحسوبة في معادلة نيوتن

7.2.2 الفروق الخلفية (Backward Differences):

تُستخدم أيضًا عندما تكون النقاط متساوية التباعد، ولكن تعتمد على الفروق من النهاية، وشكل الدالة هو:

$$p(x) = y_n + \ \nabla_{y_n}(x - x_n) + \frac{\nabla^{\,2}y_n(x-x_n)(x-x_{n-1})}{2!} + \cdots$$

- الفرق الخلفي الأول: $\nabla_{y_n} = y_n - y_{n-1}$
- الفرق الخلفي الثاني: $\nabla_{y_n} = y_{n-1} - y_{n-2}$

طريقة الحل:

1. بناء جدول الفروق الخلفية
2. حساب الفرق الخلفي الأول، ثم الثاني، وهكذا
3. استخدام القيم المحسوبة في معادلة نيوتن

7.2.3 الفروق المقسومة (Divided Differences):

تُستخدم مع النقاط غير متساوية التباعد، وشكل الدالة هو:

$$p(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_{1,}, x_2](x - x_0)(x - x_1) + \cdots$$

- القيمة الأولى: $f[x_0]$
- الفرق المقسوم الأول: $f[x_0, x_1] = \frac{f[x_1]-f[x_0]}{x_1-x_0}$
- الفرق المقسوم الثاني: $f[x_0, x_{1,}, x_2] = \frac{f[x_1,x_2]-f[x_0,x_1]}{x_2-x_0}$

طريقة الحل:

1. حساب جدول الفروق المقسومة
2. إيجاد الفروق الأولى، ثم الثانية، وهكذا
3. استخدام القيم المحسوبة في معادلة نيوتن

مثال نظري لطريقة الفروق الأمامية ثم نطبقه برمجياً:

لدينا الجدول التالي للقيم:

x	1	2	3	4
$f(x)$	2	5	10	17

نريد إيجاد قيمة f(2.5)

الخطوات:

1. نحسب فروق القيم $\Delta f(x)$:

- $\Delta f_1 = f(2) - f(1) = 5 - 2 = 3$
- $\Delta f_2 = f(3) - f(2) = 10 - 5 = 5$
- $\Delta f_3 = f(4) - f(3) = 17 - 10 = 7$

2. نحسب الفروق الثانية $\Delta^2 f(x)$:

- $\Delta^2 f_1 = \Delta f_2 - \Delta f_1 = 5 - 3 = 2$
- $\Delta^2 f_2 = \Delta f_3 - \Delta f_2 = 7 - 5 = 2$

3. نحسب الفروق الثالثة $\Delta^2 f(x)$:

- $\Delta^3 f_1 = \Delta^2 f_2 - \Delta^2 f_1 = 2 - 2 = 0$

4. نطبق صيغة نيوتن للفروق الأمامية:

$$p(x) = f_1 + r\Delta f_1 + \frac{r(r-1)}{2!}\Delta^2 f_1 + \frac{r(r-1)(r-2)}{3!}\Delta^3 f_1$$

التطبيق برمجياً:

```
import sympy as sp
# تعريف المتغيرات
x = sp.symbols('x')
data_x = [1, 2, 3, 4]  # قيم x
data_y = [2, 5, 10, 17]  # قيم f(x)
h = data_x[1] - data_x[0]  # الفرق بين نقاط x
# حساب الفروق الأمامية
def forward_differences(y_values):
    n = len(y_values)
    differences = [y_values]
    for i in range(1, n):
        prev_diff = differences[-1]
        new_diff = [prev_diff[j+1] - prev_diff[j] for j in
range(len(prev_diff)-1)]
        differences.append(new_diff)
    return differences
# استخراج الفروق الأمامية
differences = forward_differences(data_y)
# صياغة استيفاء نيوتن
u = (x - data_x[0]) / h
P = data_y[0]
factorial = 1
u_product = 1
for i in range(1, len(differences)):
    u_product *= (u - (i - 1))
    factorial *= i
    P += (u_product / factorial) * differences[i][0]
# تبسيط الدالة
P_simplified = sp.simplify(P)
print("دالة الاستيفاء: ", P_simplified)
# إيجاد f(2.5)
result = P_simplified.subs(x, 2.5)
print("قيمة f(2.5): ", result)
```

مثال نظري لطريقة الفروق الخلفية ثم نطبقه برمجياً:

لدينا الجدول التالي للقيم:

x	1	2	3	4
$f(x)$	2	5	10	17

نريد إيجاد قيمة f(2.5)

الخطوات:

1. نحسب فروق القيم الخلفية $\nabla f(x)$:

- $\nabla f_1 = f(2) - f(1) = 5 - 2 = 3$
- $\nabla f_2 = f(3) - f(2) = 10 - 5 = 5$
- $\nabla f_3 = f(4) - f(3) = 17 - 10 = 7$

2. نحسب فروق القيم الخلفية الثانية $\nabla^2 f(x)$:

- $\nabla^2 f_1 = \nabla f_2 - \nabla f_1 = 5 - 3 = 2$
- $\nabla^2 f_2 = \nabla f_3 - \nabla f_2 = 7 - 5 = 2$

3. نحسب الفروق الثالثة $\nabla^3 f(x)$:

- $\nabla^3 f_1 = \nabla^2 f_2 - \nabla^2 f_1 = 2 - 2 = 0$

4. نطبق صيغة نيوتن للفروق الأمامية:

$$p(x) = f_1 + r\nabla f_1 + \frac{r(r-1)}{2!}\nabla^2 f_1 + \frac{r(r-1)(r-2)}{3!}\nabla^3 f_1$$

التطبيق برمجياً:

```
import sympy as sp
# تعريف المتغيرات
x = sp.symbols('x')
data_x = [1, 2, 3, 4]  # قيم x
data_y = [2, 5, 10, 17]  # قيم f(x)
h = data_x[1] - data_x[0]  # الفرق بين نقاط x
# حساب الفروق الخلفية
def backward_differences(y_values):
    differences = y_values[-1:]  # بداية الفروق من آخر عنصر
    for i in range(len(y_values)-1, 0, -1):
        prev_diff = differences
        new_diff = [(prev_diff[j] - prev_diff[j-1]) for j in
range(len(prev_diff)-1, 0, -1)]  # الفرق الخلفي
        differences = new_diff
    return differences
 # استخراج الفروق الخلفية
differences = backward_differences(data_y)
# صياغة استيفاء نيوتن
u = (x - data_x[0]) / h
P = data_y[0]
factorial = 1
u_product = 1
for i in range(1, len(differences)):
    u_product *= (u - (i - 1))
    factorial *= i
    P += (u_product / factorial) * differences[i][0]
# تبسيط الدالة
P_simplified = sp.simplify(P)
print("دالة الاستيفاء: ", P_simplified)
# إيجاد f(2.5)
result = P_simplified.subs(x, 2.5)
print("قيمة f(2.5): ", result)
```

مثال نظري لطريقة الفروق المقسومة ثم نطبقه برمجياً:

لدينا الجدول التالي للقيم:

x	1	2	4	7
$f(x)$	3	6	8	10

نريد إيجاد قيمة f(2.5)

الخطوات:

1. نحسب الفروق المقسومة بين كل نقطتين متتاليتين:

- $f[x_0, x_1] = \frac{f[x_1]-f[x_0]}{x_1-x_0} = \frac{6-3}{2-1} = 3$
- $f[x_0, x_1, x_2] = \frac{f[x_1,x_2]-f[x_0,x_1]}{x_2-x_0} = \frac{1-3}{4-1} = \frac{-2}{3}$
- $f[x_0, x_1, x_2, x_3] = \frac{f[x_1,x_2,x_3]-f[x_0,x_1,x_2]}{x_3-x_0} = \frac{1}{10}$

2. نطبق صيغة نيوتن للفروق المقسومة:

$$p(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + f[x_0, x_1, x_2, x_3](x - x_0)(x - x_1)(x - x_2)$$

$$p(2.5) = 3 + 4.5 - 0.5 - 0.1125 = 6.8875$$

التطبيق برمجياً:

```
import sympy as sp
# تعريف المتغيرات
x = sp.symbols('x')
# إدخال القيم
data_x = [1, 2, 4, 7]
data_y = [3, 6, 8, 10]
# حساب جدول الفروق المقسومة
def divided_differences(x_values, y_values):
    n = len(x_values)
    table = [y_values]  # الأصلية y الصف الأول هو قيم
    for level in range(1, n):
        current_differences = []
        for i in range(n - level):
            diff = (table[level - 1][i + 1] - table[level - 1][i]) /
(x_values[i + level] - x_values[i])
            current_differences.append(diff)
        table.append(current_differences)
    return table
# بناءً على الشكل العام P(x) حساب قيمة
def newton_interpolation(x_values, y_values, value):
    table = divided_differences(x_values, y_values)
    n = len(x_values)
    # إنشاء متعدد الحدود
    P = y_values[0]
    product = 1
    for level in range(1, n):
        product *= (x - x_values[level - 1])
        P += product * table[level][0]
    # تبسيط متعدد الحدود
    P = sp.simplify(P)
    result = P.subs(x, value)
    return P, result
# تطبيق المثال
P, result = newton_interpolation(data_x, data_y, 2.5)
print("متعدد الحدود P(x):", P)
print("قيمة P(2.5):", result)
```

8 التطبيق الملحق

قمنا ببرمجة تطبيق برمجي بلغة بايثون باستخدام مكتبة Kivy لتطوير واجهات المستخدم التفاعلية. التطبيق صُمم ليكون أداةً متعددة الوظائف في مجال التحليل العددي والرياضيات، حيث يتيح للمستخدم القيام بما يلي:

- حل المعادلات: يقدم خيارات لحل المعادلات الخطية وغير الخطية بطريقة مرنة وسهلة.
- حساب الاشتقاقات: يمكن للمستخدم إدخال أي دالة رياضية للحصول على مشتقها الأول مباشرة.
- التكامل: يتيح التطبيق حساب التكاملات غير المحددة والمحددة مع إدخال حدود التكامل.
- الاستيفاء: يدعم طرق استيفاء متعددة مثل طريقة لاغرانج وطريقة نيوتن، بالإضافة إلى خيارات للفروق الأمامية، الخلفية، والمقسومة.

8.1 ميزات التطبيق

- واجهات مستخدم بسيطة وسهلة الاستخدام تناسب جميع الفئات.
- دعم لإدخال المعادلات الرياضية والدوال بشكل مباشر باستخدام صيغة Python.
- عرض النتائج بشكل فوري مع معالجة الأخطاء في حال إدخال بيانات غير صحيحة.
- إمكانية التنقل بين الأقسام بسهولة ووضوح.
- يهدف هذا التطبيق إلى توفير أداة مساعدة للطلاب والمهتمين بالرياضيات لتحليل المشكلات وحلها بسهولة، وهو مناسب للاستخدام الأكاديمي والتعليمي.

9 الخاتمة

في ختام هذا العمل، قمنا بتقديم دراسة شاملة حول موضوع التحليل العددي وتطبيقاته البرمجية باستخدام لغة البرمجة بايثون ومكتبة SymPy. تناولنا الحلول البرمجية المختلفة لمفاهيم رياضية متقدمة مثل حل المعادلات، الاشتقاق، التكامل، والاستيفاء.

تُظهر هذه الدراسة أهمية الجمع بين الأساسيات النظرية والبرمجيات العملية لتوفير أدوات تسهّل العمليات الحسابية وتُثري التجارب التعليمية. نأمل أن يكون هذا العمل مرجعًا مفيدًا للطلاب والمبرمجين المهتمين بهذا المجال، ودعوة للمزيد من التطوير والبحث فيه.

10 المراجع

- كتاب التحليل العددي للمهندسين

- Chapra, S. C., & Canale, R. P. (2015). Numerical methods for engineers (7th ed.). McGraw-Hill Education.

- مكتبة (sympy)

- SymPy Documentation. (2024). SymPy: Python library for symbolic mathematics. https://docs.sympy.org/latest/

- مرجع (kivy framework)

- Kivy Documentation. (2024). Kivy: Cross-platform Python framework for developing multi-touch applications. https://kivy.org/doc/stable/

- مشاريع مشابهة

- https://github.com/glider4/numerical-analysis

الفهرس

1 مقدمة 1
2 نظرة عامة على التحليل العددي 2
2.1 تعريف التحليل العددي 2
2.2 أهميته وتطبيقات 2
3 نظرة عامة على مكتبة SymPy 2
3.1 الخصائص الرئيسية لمكتبة SymPy 3
3.1.1 حل المعادلات الجبرية 3
3.1.2 التفاضل والتكامل الرمزي 3
3.1.3 المصفوفات 3
3.1.4 الرسوم البيانية 3
3.1.5 دعم تنسيقات متعددة 3
3.2 التطبيقات الأكاديمية والصناعية 3
4 حل المعادلات الرياضية باستخدام SymPy 4
4.1 حل المعادلات الخطية 4
4.2 حل المعادلات غير الخطية 5
5 الاشتقاق والتفاضل 5
5.1 اشتقاق دالة بمتغير واحد 6
5.2 اشتقاق دالة بعدة متغيرات 6
5.3 المشتقات العليا 7
6 التكامل 8
6.1 التكامل غير المحدد 8
6.2 التكامل المحدد 9
7 الاستيفاء Interpolation 10
7.1 استيفاء لاغرانج Lagrange Interpolation 10
7.2 استيفاء نيوتن Newton Interpolation 11
7.2.1 الفروق الأمامية (Forward Differences) 11
7.2.2 الفروق الخلفية (Backward Differences) 11
7.2.3 الفروق المقسومة (Divided Differences) 12
8 التطبيق الملحق 19
8.1 ميزات التطبيق 19
9 الخاتمة 20
10 المراجع 21